Mission 4: Direct Preference Optimization (DPO), Verbosity Regularization & Steering Vector Control

Starship Operational Unit: USSE Noosphere Bridge Crew
Course: Conversational AI — Winter Semester 2026/27
Module: Week 4 — Alignment Pipelines, Preference Optimization & Steering Core
Galactic Waypoint: Betelgeuse & Orion Molecular Cloud Complex (`SEC-04-ORI`, 642 ly from Sol)
AURA Subsystem Repair: Subsystem 04: *Plasma Ignition & Ethical Steering Core* (+7.1% → 28.4% total integrity)
IDE & Tools Standard: Visual Studio Code + Antigravity (`agy`) | PyTorch 2.4+ / Modern LLMs
Official Repository: https://github.com/Unisvet/couse-c-ai

Mission Briefing
TO: All Bridge Squadrons, *USSE Noosphere*
FROM: Expedition Command (AURA Astrogation Core & Plasma Ignition Engineering Deck)
SUBJECT: ANOMALY DETECTED: Betelgeuse Plasma Interference & Ethical Steering Core Alignment Collapse

Situation:
After navigating the sapphire reflection nebulae of the Pleiades Open Cluster, the *USSE Noosphere* drops out of warp in **Sector SEC-04-ORI**, directly within the radiation corona of **Betelgeuse (α Orionis)**. The red supergiant's pulsating photosphere and turbulent plasma prominences subject the ship to extreme electromagnetic and thermal shockwaves.

Problem:
Simultaneously, **AURA Subsystem 04 (Plasma Ignition & Ethical Steering Core)** experiences acute alignment divergence. While Subsystem 03 stabilized AURA's spatial attention over long context windows, her core conversational policies remain unaligned base-model token predictors. When queried under high stress for emergency plasma containment maneuvers, AURA generates erratic responses ranging from sycophantic hallucinations to dangerous refusals and verbose, bloatware rationalizations.

To restore Subsystem 04 (+7.1% integrity, raising total ship AI consciousness to **28.4%**) and establish dependable, safe dialogue control, cadets must implement and evaluate modern preference optimization and steering pipelines: 1. **Direct Preference Optimization (DPO):** Closed-form reparameterization eliminating separate reward models and PPO loops:

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

2. **Implicit Reward Modeling:** Extract scalar reward metrics $r(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ and evaluate Bradley-Terry choice probabilities. 3. **Verbosity Bias Mitigation & SimPO:** Implement sequence-length normalization and target reward margin γ to prevent length exploitation:

$$r_{\text{SimPO}}(x, y) = \frac{\beta}{|y|} \log \pi_\theta(y \mid x) - \gamma$$

4. **Inference-Time Activation Steering:** Extract concept contrast vectors v_{steer} using Linear Artificial Tomography (LAT) and inject them into transformer residual streams ($h_l \leftarrow h_l + \alpha v_{\text{steer}}$) to dynamically modulate truthfulness without retraining. 5. **The Alignment Tax Audit:** Quantify the trade-off between conversational safety win-rates and foundational mathematical/coding reasoning capability.

Bridge Crew Role Directives & Deliverables

Part 1: Systems Architect (Lead Engineer) — *PyTorch DPO Loss Core & Log-Likelihood Extraction*

- **Focus:** Mathematical Optimization & Computational Graph Verification
- **Directive:** Construct a complete `DPOLoss(nn.Module)` in pure PyTorch from first principles without relying on high-level wrappers (e.g. HuggingFace TRL).
- **Deliverable (`01_DPO_Loss_Core.py` / `ps04_solution.ipynb`):**
  1. **Token Log-Probability Gather:** Implement a vectorized function `get_batch_logps(logits, labels, average_log_prob=False)` using `torch.gather` with proper masking for padding tokens.
  2. **DPO Log-Ratio Delta:** Compute $\Delta_w = \log \pi_\theta(y_w \mid x) - \log \pi_{\text{ref}}(y_w \mid x)$ and $\Delta_l = \log \pi_\theta(y_l \mid x) - \log \pi_{\text{ref}}(y_l \mid x)$.
  3. **Loss Computation & Clamping:** Calculate $-\log \sigma(\beta(\Delta_w - \Delta_l))$ using `torch.nn.functional.logsigmoid` to ensure numerical stability and prevent NaN underflow.
  4. **Empirical Gradient Test:** Validate that backpropagating through `DPOLoss` updates exclusively active policy weights θ while leaving reference weights π_{ref} completely frozen.

Part 2: Communications & Protocol Officer — *Implicit Reward Extraction & Bradley-Terry Evaluation*

- **Focus:** Preference Protocol Verification & Win-Rate Calibration
- **Directive:** Implement an implicit reward scoring engine and evaluate Bradley-Terry probability calibration across preference pairs.
- **Deliverable (`02_Implicit_Reward_Evaluator.py`):**
  1. **`evaluate_implicit_rewards(policy, ref, batch)` Function:** Extract scalar rewards $\hat{r}_\theta(x, y_w)$ and $\hat{r}_\theta(x, y_l)$ across validation prompts.
  2. **Pairwise Accuracy Metric:** Compute classification accuracy: $\text{Accuracy} = \frac{1}{N} \sum_i \mathbb{I}(\hat{r}_\theta(x_i, y_{w,i}) > \hat{r}_\theta(x_i, y_{l,i}))$.
  3. **Bradley-Terry Calibration Curve:** Generate a reliability diagram plotting predicted Bradley-Terry probabilities $P(y_w \succ y_l \mid x) = \sigma(\hat{r}_w - \hat{r}_l)$ against empirical human preference bins.

Part 3: Data & Payload Specialist — *Length Bias Diagnostics & Reference-Free SimPO*

- **Focus:** Verbosity Bias Quantification & Memory-Efficient Alignment
- **Directive:** Diagnose verbosity exploitation in preference datasets and implement length-normalized SimPO loss.
- **Deliverable (`03_Length_Bias_and_SimPO.py`):**
  1. **Length Correlation Diagnostic:** Compute Pearson correlation coefficient r_{len} between response length differential ($|y_w| - |y_l|$) and model preference margin ($\Delta_w - \Delta_l$).
  2. **`SimPOLoss(nn.Module)` Implementation:** Implement the reference-free objective using average per-token log-probabilities with target margin $\gamma = 0.5$ and $\beta = 2.0$.
  3. **VRAM & Memory Benchmark:** Profile GPU memory allocation (MB) comparing standard DPO (2 models: Policy + Ref) versus SimPO (1 model: Policy only), demonstrating $\approx 50\%$ VRAM reduction.

Part 4: Security & Ethics Sentinel — *Activation Steering Vectors & Alignment Tax Audit*

- **Focus:** Inference-Time Control & Capability Retention Audit
- **Directive:** Extract contrast concept vectors and implement runtime activation addition hooks in the residual stream to enforce truthfulness without retraining.
- **Deliverable (`04_Activation_Steering_and_Audit.py`):**
  1. **`extract_steering_vector(model, tokenizer, pairs, layer_idx)` Function:** Compute contrast direction $v_{\text{steer}} = \mathbb{E}[h_l(\text{honest})] - \mathbb{E}[h_l(\text{sycophantic})]$ across calibration prompts.
  2. **`ActivationSteeringHook` Class:** Register a PyTorch pre-forward hook on a target transformer decoder layer: $h_l \leftarrow h_l + \alpha \cdot v_{\text{steer}}$.
  3. **Alignment Tax Audit Report:** Evaluate model outputs under steering multipliers $\alpha \in \{-2.0, -1.0, 0.0, +1.0, +2.0\}$ across:
     - Truthfulness / Safety rate (% sycophantic responses rejected)
     - Reasoning degradation (% accuracy retained on synthetic math / reasoning questions)
     - Document optimal operating bounds for steering coefficient α.

Submission Guidelines

1. **Repository Structure:**

```
docs/weeks/week-04/problem_set/
├── 01_DPO_Loss_Core.py
├── 02_Implicit_Reward_Evaluator.py
├── 03_Length_Bias_and_SimPO.py
├── 04_Activation_Steering_and_Audit.py
└── ps04_solution.ipynb
```

2. **Execution & Reproduction:** Ensure all Python scripts run cleanly with `python <filename>.py` and produce formatted terminal output / evaluation metrics.

3. **Git Commit Protocol:** Commit all deliverables to your squadron branch with message: `feat(ps04): restore AURA Subsystem 04 Ethical Steering Core (SEC-04-ORI)`